

【报数游戏】

题目描述

100个人围成一圈，每个人有一个编码，编号从1开始到100。

他们从1开始依次报数，报到为M的人自动退出圈圈，然后下一个人接着从1开始报数，直到剩余的人数小于M。

请问最后剩余的人在原先的编号为多少？

输入描述

输入一个整数参数 M

输出描述

如果输入参数M小于等于1或者大于等于100，输出“ERROR!”；

否则按照原先的编号从小到大的顺序，以英文逗号分割输出编号 **字符串**

用例

输入	3
输出	58,91
说明	输入M为3，最后剩下两个人。

输入	4
输出	34,45,97

说明	输入M为4，最后剩下三个人。
----	----------------

题目解析

本题是经典的 **约瑟夫环问题**，约瑟夫环主要是模拟环，这里的环可以使用循环链表，双端队列模拟，也可以基于数组结构，使用特定逻辑模拟环向运动。

但是最简单最通用的还是使用双端队列。

Java算法源码

```
1 import java.util.ArrayDeque;
2 import java.util.Scanner;
3 import java.util.StringJoiner;
4
5 public class Main {
6     public static void main(String[] args) {
7         Scanner sc = new Scanner(System.in);
8
9         int m = sc.nextInt();
10
11         // 如果输入参数M小于等于1或者大于等于100，输出“ERROR!”
12         if (m <= 1 || m >= 100) {
13             System.out.println("ERROR!");
14             return;
15         }
16
17         // 使用双端队列模拟约瑟夫环
18         ArrayDeque<Integer> queue = new ArrayDeque<>();
19         for (int id = 1; id <= 100; id++) queue.addLast(id);
20
21         int num = 1;
22         while (queue.size() >= m) {
23             // 编号id的人报数num
24             int id = queue.pollFirst();
```

```

25 |         if (num == m) {
26 |             num = 1;
27 |         } else {
28 |             num++;
29 |             queue.addLast(id);
30 |         }
31 |     }
32 | }
33 |
34 |     StringJoiner sj = new StringJoiner(",");
35 |     queue.stream().sorted().forEach(id -> sj.add(id + ""));
36 |
37 |     System.out.println(sj);
38 | }
39 | }

```

JS算法源码

```

1 | const rl = require("readline").createInterface({ input: process.stdin });
2 | var iter = rl[Symbol.asyncIterator]();
3 | const readline = async () => (await iter.next()).value;
4 |
5 | void (async function () {
6 |     const m = parseInt(await readline());
7 |
8 |     // 如果输入参数M小于等于1或者大于等于100, 输出“ERROR!”
9 |     if (m <= 1 || m >= 100) {
10 |         console.log("ERROR!");
11 |         return;
12 |     }
13 |
14 |     // 使用数组模拟双端队列, 使用双端队列模拟约瑟夫环
15 |     const queue = new Array(100).fill(0).map( (_, i) => i + 1);
16 |

```

运行

```

17 | let num = 1;18 | while (queue.length >= m) {
19 |     // 编号id的人报数num
20 |     const id = queue.shift();
21 |
22 |     if (num == m) {
23 |         num = 1;
24 |     } else {
25 |         num++;
26 |         queue.push(id);
27 |     }
28 | }
29 |
30 | console.log(queue.sort((a, b) => a - b).join(","));
31 | })();

```

Python算法源码

```

1 | from collections import deque
2 |
3 | # 输入获取
4 | m = int(input())
5 |
6 |
7 | # 核心代码
8 | def solution():
9 |     if m <= 1 or m >= 100:
10 |         return "ERROR!"
11 |
12 |     que = deque([i for i in range(1, 101)])
13 |
14 |     num = 1
15 |     while len(que) >= m:
16 |         id = que.popleft()
17 |

```

```

18         if num == m:
19             num = 1
20         else:
21             num += 1
22             que.append(id)
23
24     lst = list(que)
25     lst.sort()
26     return ",".join(map(str, lst))
27
28
29 # 输出打印
30 print(solution())

```

C算法源码

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <stdbool.h>
4
5  typedef struct ListNode {
6      int ele;
7      struct ListNode *next;
8  } ListNode;
9
10 typedef struct LinkedList {
11     int size;
12     ListNode *head;
13     ListNode *tail;
14 } LinkedList;
15
16 LinkedList *new_LinkedList() {
17     LinkedList *link = (LinkedList *) malloc(sizeof(LinkedList));
18     link->size = 0;
19     link->head = NULL;

```

```
20 |     link->tail = NULL;
    |           21 |     return link;
22 | }
23 |
24 | void addLast_LinkedList(LinkedList *link, int ele) {
25 |     ListNode *node = (ListNode *) malloc(sizeof(ListNode));
26 |     node->ele = ele;
27 |     node->next = NULL;
28 |
29 |     if (link->size == 0) {
30 |         link->head = node;
31 |         link->tail = node;
32 |     } else {
33 |         link->tail->next = node;
34 |         link->tail = node;
35 |     }
36 |
37 |     link->size++;
38 | }
39 |
40 | int removeFirst_LinkedList(LinkedList *link) {
41 |     if (link->size == 0) exit(-1);
42 |
43 |     ListNode *removed = link->head;
44 |
45 |     if (link->size == 1) {
46 |         link->head = NULL;
47 |         link->tail = NULL;
48 |     } else {
49 |         link->head = link->head->next;
50 |     }
51 |
52 |     link->size--;
53 |
54 |     return removed->ele;
55 | }
56 |
```

```

57 int main() {58     int m;
59     scanf("%d", &m);
60
61     // 如果输入参数M小于等于1或者大于等于100, 输出“ERROR!”
62     if (m <= 1 || m >= 100) {
63         puts("ERROR!");
64         return 0;
65     }
66
67     // 使用双端队列模拟约瑟夫环
68     LinkedList *queue = new_LinkedList();
69     for (int i = 1; i <= 100; i++) addLast_LinkedList(queue, i);
70
71     // leave[i] 表示 编号i 的人是否退出
72     bool leave[101] = {false};
73
74     int num = 1;
75     while (queue->size >= m) {
76         // 编号id的人报数num
77         int id = removeFirst_LinkedList(queue);
78
79         if (num == m) {
80             num = 1;
81             leave[id] = true;
82         } else {
83             num++;
84             addLast_LinkedList(queue, id);
85         }
86     }
87
88     for (int i = 1; i <= 100; i++) {
89         if (!leave[i]) {
90             printf("%d", i);
91
92             if (--queue->size > 0) {
93                 printf(",");
94             }

```

```
95 |         }
96 |     }
97 |
98 |     return 0;
99 | }
```

C++算法源码

```
1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  int main() {
6      int m;
7      cin >> m;
8
9      // 如果输入参数M小于等于1或者大于等于100, 输出“ERROR!”
10     if (m <= 1 || m >= 100) {
11         cout << "ERROR!" << endl;
12         return 0;
13     }
14
15     // 使用双端队列模拟约瑟夫环
16     deque<int> queue;
17     for (int i = 1; i <= 100; i++) queue.push_back(i);
18
19     int num = 1;
20     while (queue.size() >= m) {
21         // 编号id的人报数num
22         int id = queue.front();
23         queue.pop_front();
24
25         if (num == m) {
26             num = 1;
27         } else {
```

复制

```
28         num++;
29         queue.push_back(id);
30     }
31 }
32
33 sort(queue.begin(), queue.end());
34
35 for (int i = 0; i < queue.size(); i++) {
36     cout << queue[i];
37
38     if (i < queue.size() - 1) {
39         cout << ",";
40     }
41 }
42
43 return 0;
44 }
```